

A High Performance Hardware Implementation of SHA-256 Algorithm

Tuo Li^{1,2}, Chao Cheng², Ruiting Wang², Changhong Wang², Xiaofeng Zou², Dunshan Yu^{3*}

1. School of Software & Microelectronics, Peking University, Beijing, China

2. Shandong Inspur Artificial Intelligence Research Institute Co., Ltd., Jinan, China

3. School of Integrated Circuit, Peking University, Beijing, China

lituo@inspur.com, chengchao02@inspur.com, wangruitong@inspur.com, wangchh01@inspur.com, zouxf@inspur.com, yuds@pku.edu.cn

Corresponding Author: Dunshan Yu Email: yuds@pku.edu.cn

Abstract—SHA-256 is a well-reported algorithm extensively utilized in security applications. This paper presents a high performance hardware implementation of SHA-256 algorithm. The SHA-256 algorithm is reconstructed based on 64-level high-speed full pipeline computing architecture design to improve the algorithm throughput. In addition, the timing path of the compression function of SHA-256 is optimized to break the critical path, which increase the operating frequency of SHA-256 and further improve the throughput. Finally, Carry-Save Adder (CSA) technology is employed in the message expansion to further improve the timing and related parameters. The proposed architecture of SHA-256 is implemented on Xilinx Virtex-7 FPGA device, and the experimental results indicate that the designed SHA-256 demonstrates a frequency of 429 MHz, a throughput of 219.65 Gbps, and an efficiency of 24.84 Mbps/slice.

Keywords—SHA-256, full pipeline, path optimization, high performance

I. INTRODUCTION

The secure hash algorithms are employed widely in security scenarios such as message authentication, digital signatures, and blockchain computing [1-4]. Currently, among prevalent SHA family hash algorithms, SHA-1 is gradually being discontinued owing to potential vulnerabilities and SHA-3 is not widely used in the industry yet. SHA-256, as one standard of the SHA-2 family algorithms, is the most popular hash algorithm to implement secure tasks, including file integrity verification, digital signature verification, and firmware verification.

A widely reported fact is that hardware implementation of hash algorithms is more efficient and secure than software implementation [5-6]. Therefore, Hash hardware accelerators are available for large-scale data stream processing and fast encryption security scenarios. However, standard SHA-256 hardware accelerators are still limited by performance bottlenecks such as low throughput and low operating frequency.

In previous literatures, various techniques have been developed to enhance the performance of the SHA-256

algorithm. Chen et al. proposed an architecture with optimized the critical path of the compression function, which increased the operating frequency [7-9]. However, it has weakness of low throughput and large computational delay for security computing of large-scale data stream. Zhang et al. proposed a full pipeline architecture to improve the throughput of SHA-256 [10]. However, the lower operating frequency limits the algorithm performance.

In this paper, a high-performance SHA-256 algorithm was provided for hashing computing of large-scale data. The SHA-256 algorithm is reconstructed based on a high-speed full pipeline computing architecture to improve throughput. In addition, the core compression function module in SHA-256 algorithm is path-optimized with method of breaking critical path and message expander is path-optimized with CSA to solve the problem of limited operating frequency. Combined with the above techniques, the optimized SHA-256 algorithm provided by this paper demonstrates excellent performance of high frequency and high throughput.

II. SHA-256 ALGORITHM

A. Message preprocessing

Message preprocessing is to pad the message before hash computing to guarantee that its length is a multiple of 512 bits. Fig. 1 shows the schematic of the message padding of SHA-256. The SHA-256 message padding process involves appending a bit “1”, followed by k bits “0”, to make the message length modulo 512 remainder 448. Finally, 64 bits of data representing the length of message are appended to the preprocessed message.

B. Message expansion

Message expansion is used to produce the message word W_i required in the calculation process of the compression function. The first 16 message words $W_0 \sim W_{15}$ are directly obtained by grouping the message, and the remaining message words $W_{16} \sim W_{63}$ are given by the following equation (1).

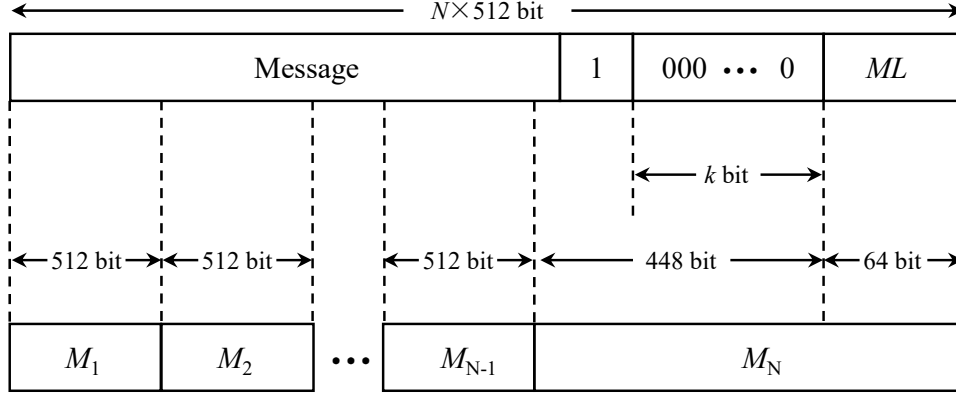


Fig. 1. Schematic of data preprocessing of SHA-256.

$$w_t = \sigma_1(w_{t-2}) + w_{t-7} + \sigma_0(w_{t-15}) + w_{t-16} \quad (1)$$

C. Iterative compression

The initial variables $H_0 \sim H_7$ are assigned to $a_0 \sim h_0$, and calculated for 64 rounds as follows, where $\sum_0, \sum_1, Maj, Ch, K_t, W_t$ are given in [8].

$$\begin{cases} a_{t+1} = \sum_0(a_t) + Maj(a_t, b_t, c_t) + \sum_1(e_t) \\ \quad + Ch(e_t, f_t, g_t) + h_t + K_t + W_t \\ b_{t+1} = a_t \\ c_{t+1} = b_t \\ d_{t+1} = c_t \\ e_{t+1} = d_t + \sum_1(e_t) + Ch(e_t, f_t, g_t) \\ \quad + h_t + K_t + W_t \\ f_{t+1} = e_t \\ g_{t+1} = f_t \\ h_{t+1} = g_t \end{cases} \quad (2)$$

Subsequently, the updated intermediate hash values can be derived using equation (3).

$$\begin{cases} H_0^{(n)} = H_0^{(n-1)} + a_{64} \\ H_1^{(n)} = H_1^{(n-1)} + b_{64} \\ \vdots \\ H_7^{(n)} = H_7^{(n-1)} + h_{64} \end{cases} \quad (3)$$

III. PROPOSED DESIGN OF SHA-256

A. High-speed pipeline computing architecture design

The schematic in Fig. 2 depicts the high-speed fully pipelined architecture of the SHA-256 algorithm, fundamentally encompassing two primary components, which are a compression section and an expansion section. More specifically, SHA-256 requires 64 rounds of compression function calculations, thus the compression part in the high-speed pipeline architecture consists of 64 compressors and 64 sets of calculation variable registers. 64 compressors and 64 registers are connected in sequence to perform compression function calculations

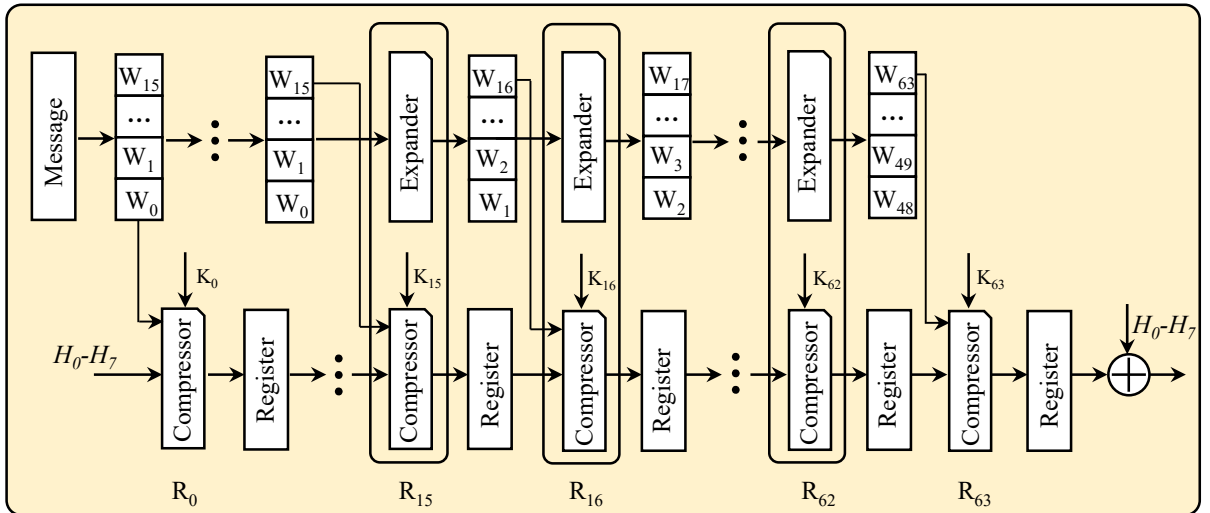


Fig. 2. Schematic of the high-speed pipeline computing architecture of SHA-256.

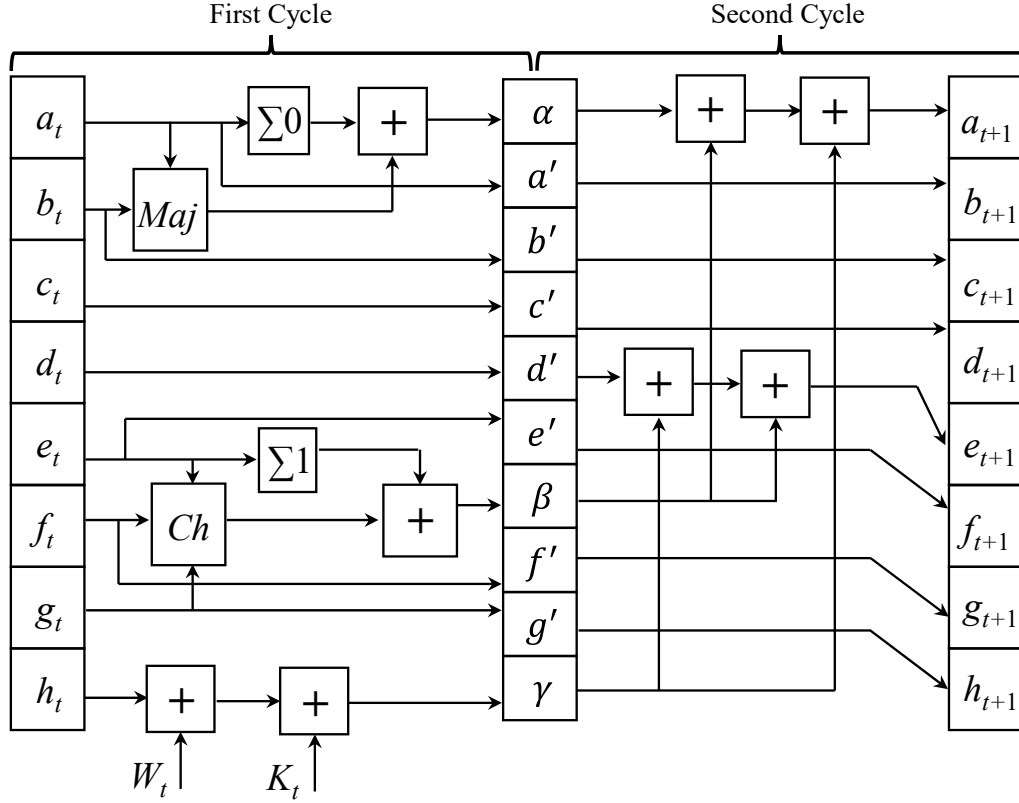


Fig. 3. Schematic of the optimized compression function of SHA-256.

simultaneously, realizing pipeline calculation and high throughput.

In addition, 64 message words need to be used in 64 rounds of iterative calculations. Since the first 16 message words are obtained by grouping the message directly, and the following 48 message words need to be calculated by message word expansion function. Therefore, the expansion part is composed of 48 expanders and a group of 16 message word registers. When the message word expander is executed, the values in the 16 message word registers are updated, and the latest calculated message word is transmitted to the compressor for calculation.

In order to ensure the execution of the pipeline, the calculation of the message word expansion is two clock cycles earlier than the compression function calculation, namely the message word W_i used in the ($i > 15$) round of compressor is obtained by the expander in the previous round computing units. After completing 64 rounds of pipeline calculation, the result of the last round of compression function calculation is added to the initial variables $H_0 \sim H_7$ to obtain the final hash digest value.

B. Path optimization design

Although the pipeline computing architecture is adopted to enhance the throughput of SHA-256 algorithm, the critical path of the compression function is complex, which seriously limits the operating frequency of SHA-256 algorithm. Therefore, the compression function

module is optimized with the method of inserting intermediate registers to shorten the critical path of algorithm, as illustrated in Fig. 3. Consequently, the optimized compression function takes two clock cycles to complete the calculation.

More specifically, $a_t \sim h_t$ are the input registers of the compression function, $a_{t+1} \sim h_{t+1}$ are the output registers of the compression function. $\alpha, \beta, \gamma, a' \sim g'$ are intermediate registers, and the calculation paths from the input registers to the intermediate registers are given by (4).

$$\left\{ \begin{array}{l} \alpha = \sum_0(a_t) + Maj(a_t, b_t, c_t) \\ a' = a_t \\ b' = b_t \\ c' = c_t \\ d' = d_t \\ e' = e_t \\ \beta = \sum_1(e_t) + Ch(e_t, f_t, g_t) \\ f' = f_t \\ g' = g_t \\ \gamma = h_t + K_t + W_t \end{array} \right. \quad (4)$$

The calculation paths from the intermediate registers to the output registers are given by (5). Through the above path optimization, the original combinational logic delays

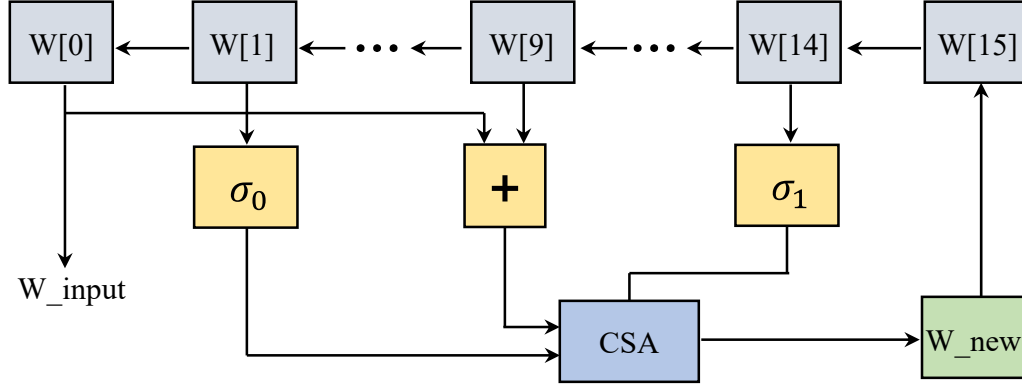


Fig. 4. Schematic of the optimized message word expansion of SHA-256.

of the output register a_{t+1} and the output register e_{t+1} are reduced, thereby improving the operating frequency.

$$\begin{cases} a_{t+1} = \alpha + \beta + \gamma \\ b_{t+1} = a' \\ c_{t+1} = b' \\ d_{t+1} = c' \\ e_{t+1} = d' + \beta + \gamma \\ f_{t+1} = e' \\ g_{t+1} = f' \\ h_{t+1} = g' \end{cases} \quad (5)$$

C. Optimization of message expansion

The calculation process of the message word expansion is given by (1), where the operations involved in σ_0 and σ_1 are shift and XOR, which will only cause lower logic gate delay. However, its timing path involves the use of three adders, which will cause a large delay. Carry-Save Adder (CSA) is the most efficient adder for combining three or more operands, enabling rapid arithmetic computation in register-transfer level (RTL) design. Therefore, the CSA is employed to implement the addition of multiple numbers during the message word expansion process to obtain the new message word, as illustrated in Fig. 4, and then the 16 message word

registers are updated in sequence.

IV. EXPERIMENTAL RESULTS

The SHA-256 design presented in this paper has been implemented on the Xilinx Virtex-7 xc7vx690tffg 1930-3 FPGA based on Vivado 19.1 tool. The hardware implementation consists of four parts, which are message preprocessing, message word expansion, compression function, and top-level integration. According to the timing report after implementation, the maximum operating frequency $f_{\max}$ can be calculated by following.

$$f_{\max} = \frac{1}{T - WNS} \quad (6)$$

And then the throughput and efficiency are given by equation (7) and (8). The SHA-256 proposed in this paper demonstrates an area utilization of 8842 slices, with an efficiency of 24.84 Mbps/slice.

$$\text{throughput} = \frac{\text{Block_size} \times f_{\max}}{\text{cycles}} \quad (7)$$

$$\text{efficiency} = \frac{\text{throughput}}{\text{slices}} \quad (8)$$

The performance comparison of our design with the counterparts reported in previous literatures is shown in Table I. The architecture of SHA-256 in this paper

TABLE I. COMPARISONS OF PERFORMANCE PARAMETERS WITH DIFFERENT SHA-256 DESIGNS.

Design	Device	Clock frequency (MHz)	Throughput (Gbps)	Area (Slices)	Efficiency (Mbps/Slice)
Padhi [7]	Virtex-4	171	1.35	610	2.20
Rote [8]	Virtex-6	271	2.04	905	2.25
Chen [9]	Virtex-4	256	1.98	979	2.03
Zhang [10]	KintexUltraScale	200	102.40	4891	20.94
This work	Virtex-7	429	219.65	8842	24.84

demonstrates the highest throughput with 219.65 Gbps and the highest frequency of 429 MHz, which realize a 115% performance improvement over the highest performance SHA-256 implementation in [10]. In addition, our design yields an outstanding efficiency of 24.84 Mbps/slice, increasing by 19% compared with the most efficient SHA-256 design currently in [10].

V. CONCLUSIONS

A high-performance computing architecture of SHA-256 algorithm is designed and implemented in this paper. A high-speed full pipeline parallel computing architecture is designed to accelerate the execution efficiency of the hash function and greatly improve the throughput of SHA-256 algorithm. And the critical path of core compression function is shortened to improve the timing. In addition, the path optimization is conducted on message word expansion based on CSA, further improving the operating frequency. Benefiting from the above optimization, the SHA-256 algorithm demonstrates excellent performance of high throughput and high frequency.

ACKNOWLEDGMENT

This work was supported by the Key R&D Program of Shandong Province, China (2021SFGC0801).

REFERENCES

- [1] H. Gupta and V. K. Sharma, "Role of multiple encryption in secure electronic transaction," *Int. Jour. Netw. Secur. App.*, vol. 3, no. 6, pp. 89–96, 2011.
- [2] E. Khan, M. W. El-Kharashi, F. Gebali, and M. Abd-El-Barr, "Design and performance analysis of a unified, reconfigurable HMAC-hash unit," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 54, no. 12, pp. 2683–2695, Dec. 2007.
- [3] J. Bonneau, A. Miller, J. Clark, A. Narayanan, J. A. Kroll, and E. W. Felten, "SoK: Research perspectives and challenges for bitcoin and cryptocurrencies," in *Proc. IEEE Symp. Secur. Privacy*, May 2015, pp. 104–121.
- [4] F. Wang, Y. Chen, R. Wang, A. O. Francis, B. Emmanuel, W. Zheng, and J. Chen, "An experimental investigation into the hash functions used in blockchains," *IEEE Trans. Eng. Manag.*, vol. 67, no. 4, pp. 1404–1424, Nov. 2020.
- [5] M. Kammoun, M. Elleuchi, M. Abid, and M. S. BenSaleh, "FPGA-based implementation of the SHA-256 hash algorithm", in *Proc IEEE international conference on design & test of integrated micro & nano-systems (DTS)*, 2020, pp. 1–6.
- [6] M. Kammoun, M. Elleuchi, M. Abid, and A. M. Obeid, "HW/SW architecture exploration for an efficient implementation of the secure hash algorithm SHA-256", *J. Commun. Softw. Syst.*, 2021, vol. 17, no. 2, pp. 87–96.
- [7] M. Padhi and R. Chaudhari, "An optimized pipelined architecture of SHA-256 hash function," in *Proc International Symposium on Embedded Computing and System Design (ISED)*, 2017, pp. 1–4.
- [8] M. D. Rote, N. Vijendran, and D. Selvakumar, "High performance SHA-2 core using the round pipelined technique", in *Proc IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*, 2015, pp. 1–6.
- [9] Y. Chen and S. Li, "A high-throughput hardware implementation of SHA-256 algorithm", in *Proc IEEE international symposium on circuits and systems (ISCAS)*, 2020, pp. 1–4.
- [10] Y. Zhang, Z. He, M. Wan, M. Zhan, M. Zhang, K. Peng, M. Song, and H. Gu, "A new message expansion structure for full pipeline SHA-2", *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 4, pp. 1553–1566, 2021.